



《软件测试》

第五章 软件测试的 过程管理



授课人：高利鹏

日期：2025年12月16日

目录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

学习目标

- ◆ 了解软件测试的阶段
- ◆ 了解测试需求和测试计划
- ◆ 了解测试设计和测试执行的过程
- ◆ 掌握软件缺陷分析的方法

目录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

软件测试的各个阶段

- 软件测试贯穿整个软件开发**生命周期**的过程，从需求分析到最后产品的维护，都离不开软件测试。
- 而软件测试在执行过程中也需要分为多个阶段进行，每个阶段各有特点，有些阶段虽然不是测试的主要工作体现的地方，但是却是一个成功的测试不可或缺的重要组成部分。

客户需求分析



软件测试的各个阶段

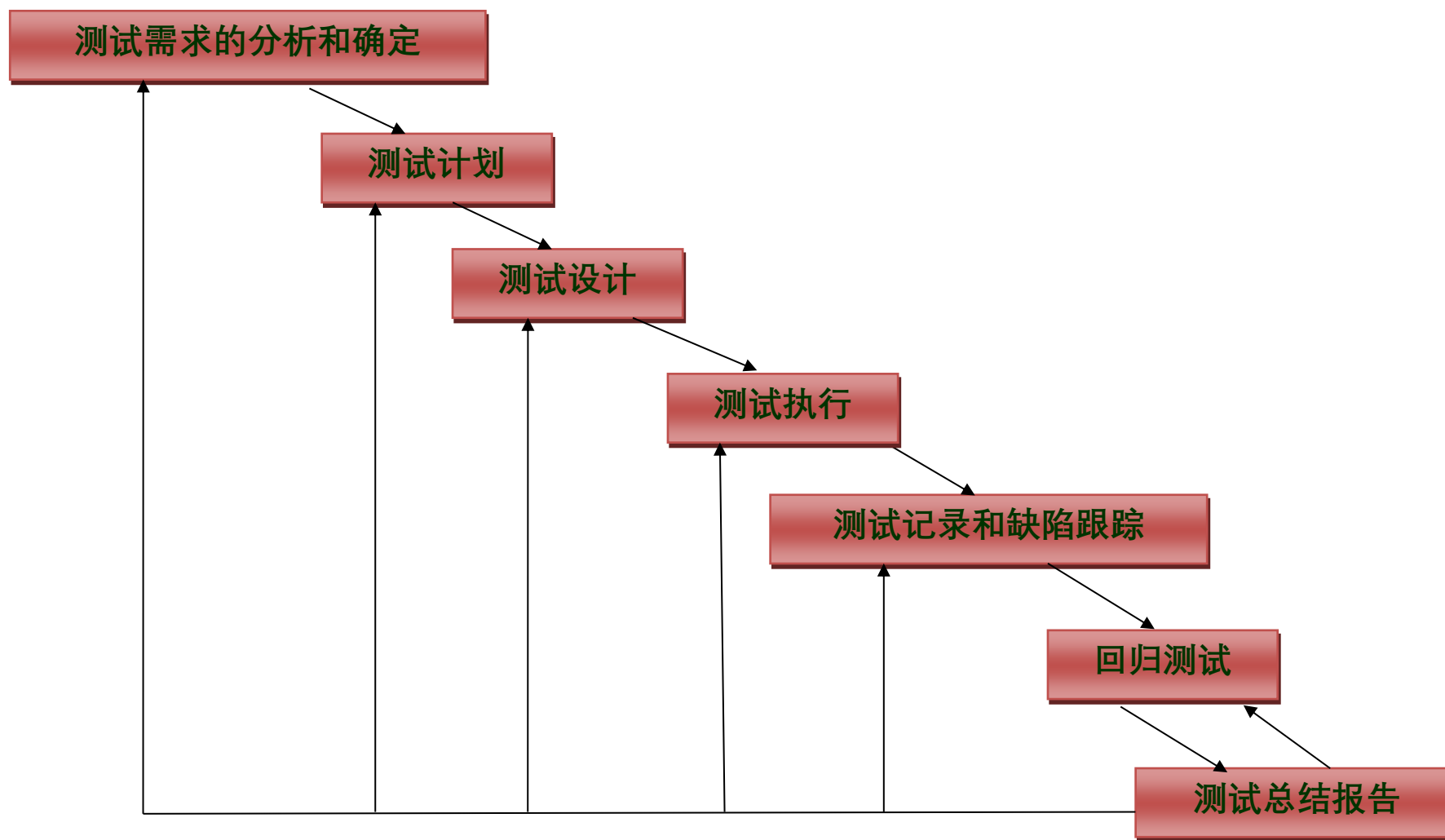
按照尽早进行测试的原则，测试人员应该在需求阶段就介入，并贯穿软件开发的全过程。就测试过程本身而言，应该包含以下几个阶段：

- 测试需求的分析和确定
- 测试计划
- 测试设计
- 测试执行
- 测试记录和缺陷跟踪
- 回归测试
- 测试总结报告

软件测试的各个阶段

- 这几个阶段可以组成一个**PDCA循环**。PDCA循环也叫戴明循环，是一种质量改进的模型。P代表计划(Plan)，D代表执行(Do)，C代表检查(Check)，A代表处理(Action)。
- 具体的流程可以概括为：**首先**在分析清楚需求的前提下对测试活动进行计划 and 设计，**然后**按既定的策划执行测试和记录测试，对测试的结果进行检查分析，形成测试报告，这些测试结果和分析报告又能指导下一步的测试设计。因此形成了一个质量改进的闭环。

软件测试的各个阶段



目 录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

目 录

CONTENTS

2.1 测试需求的分类

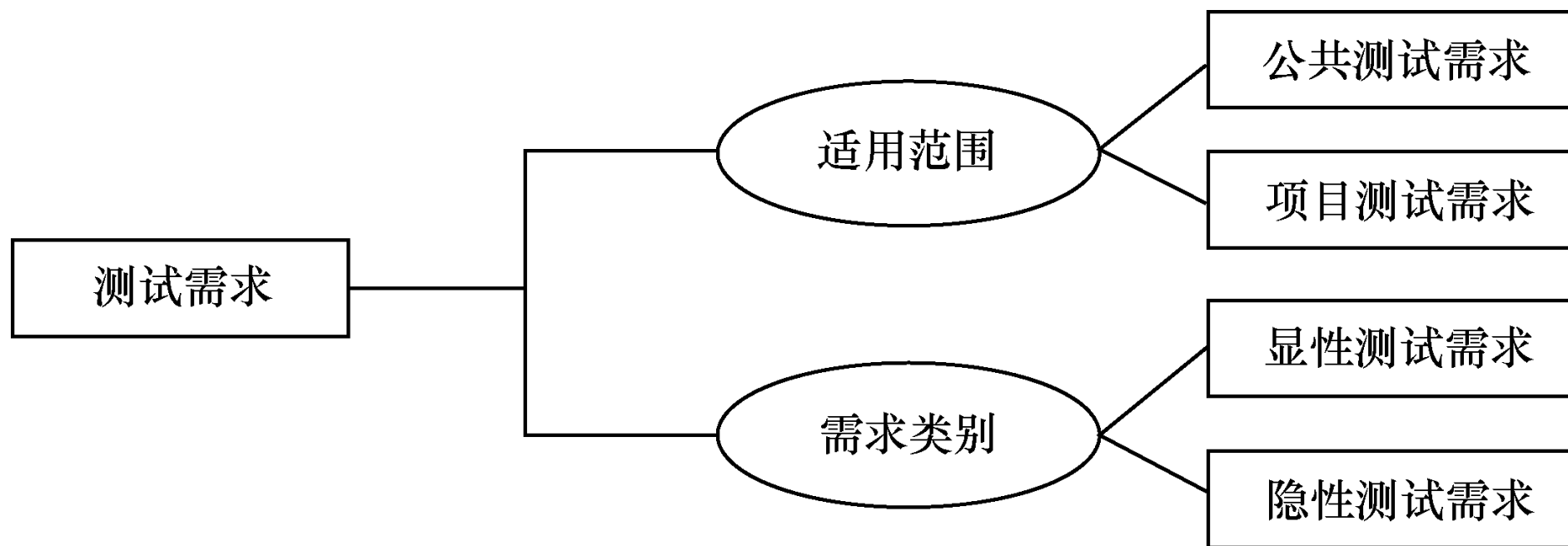
2.2 测试需求的收集

2.3 测试需求的分析

2.4 测试需求的评审

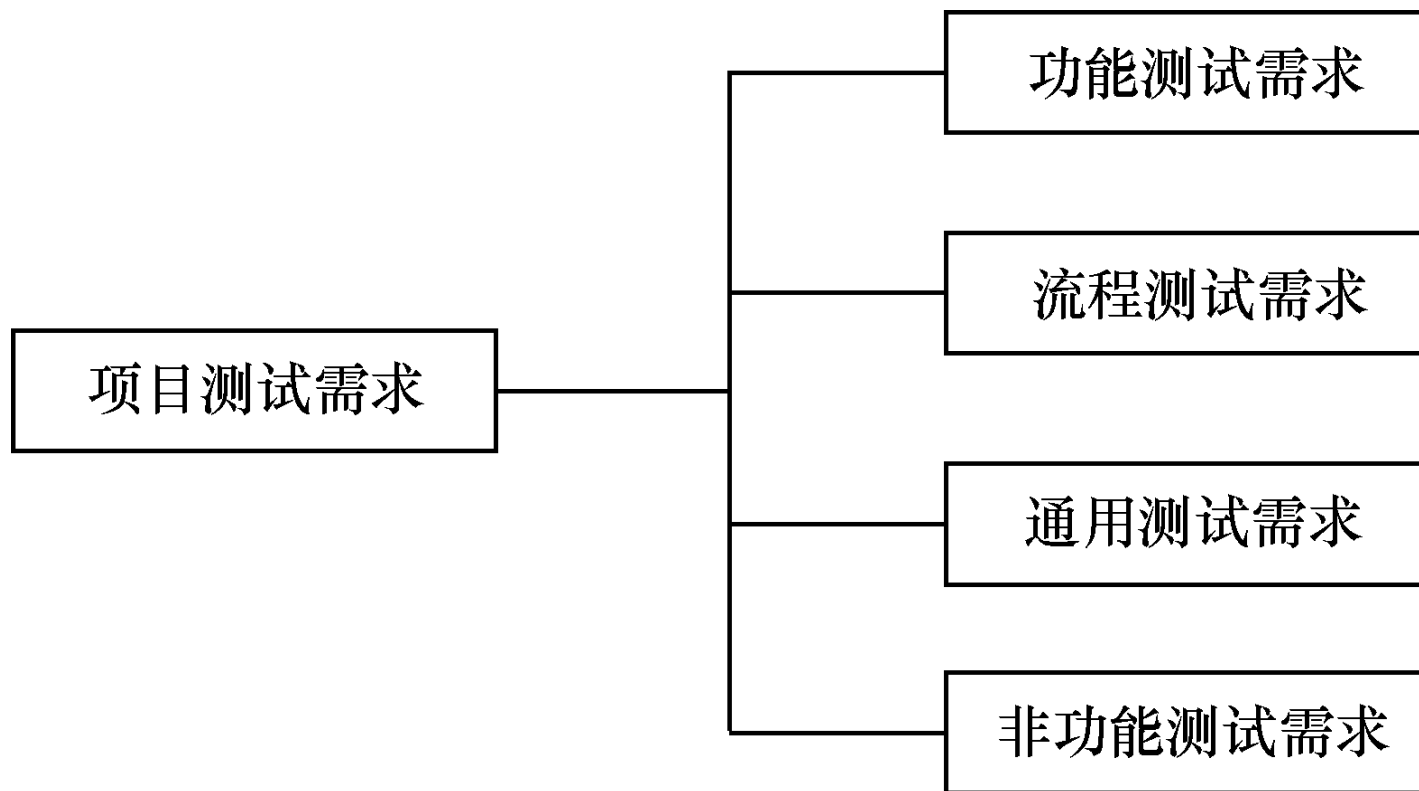
2.1 测试需求的分类

测试需求按适用范围分为公共测试需求和项目测试需求，按需求类别分为显性测试需求和隐性测试需求。



2.1 测试需求的分类

项目测试需求又分为功能测试需求、流程测试需求、通用测试需求及非功能测试需求。



2.2 测试需求的收集

软件测试需求的主要来源是**系统需求说明书（软件规格说明书）**，有了系统需求说明书基本就能画出系统。测试需求还可以通过其他途径来获得。

- （1）与待测软件相关的文档资料，如用例、界面设计、项目会议或与客户沟通时关于需求信息的会议记录、其他技术文档等。
- （2）与客户或系统分析员的沟通记录。
- （3）业务背景资料，如待测软件业务领域的知识等。
- （4）正式与非正式的培训资料。

2.3 测试需求的分析

软件需求分析、设计和实现阶段出现的问题是软件的主要错误来源。因此一旦确定软件需求，即可开始进行测试需求分析。

在收集完测试需求后，需要根据测试阶段和重点整理测试需求。

测试需求采集之后得到的是一张没有优化的需求表，需要对这份原始需求表进行初步的规划：**删除冗余重复的需求，各个需求间没有过多的交集；需覆盖业务流程、功能、非功能方面的需求。**

在做测试需求分析时需要列出以下类别：

- (1) 常用的或规定的业务流程。
- (2) 各业务流程分支的遍历。
- (3) 明确规定不可使用的业务流程。
- (4) 没有明确规定但是应该不可以执行的业务流程。
- (5) 其他异常或不符合规定的操作。

2.4 测试需求的评审

测试需求评审的内容包括**完整性审查**和**准确性审查**。

完整性审查是检查测试需求是否覆盖了所有软件需求，以及软件需求的各项特征，关注功能要求、数据定义、接口定义、性能要求、安全性要求、可靠性要求、系统约束、行业标准等，同时还要关注系统隐含的用户需求。

准确性审查是检查测试需求是否清晰、没有歧义、描述准确，是否能获得评审各方的一致理解，每项测试需求是否都可以作为设计测试用例的依据。

需求规格说明书检查要点

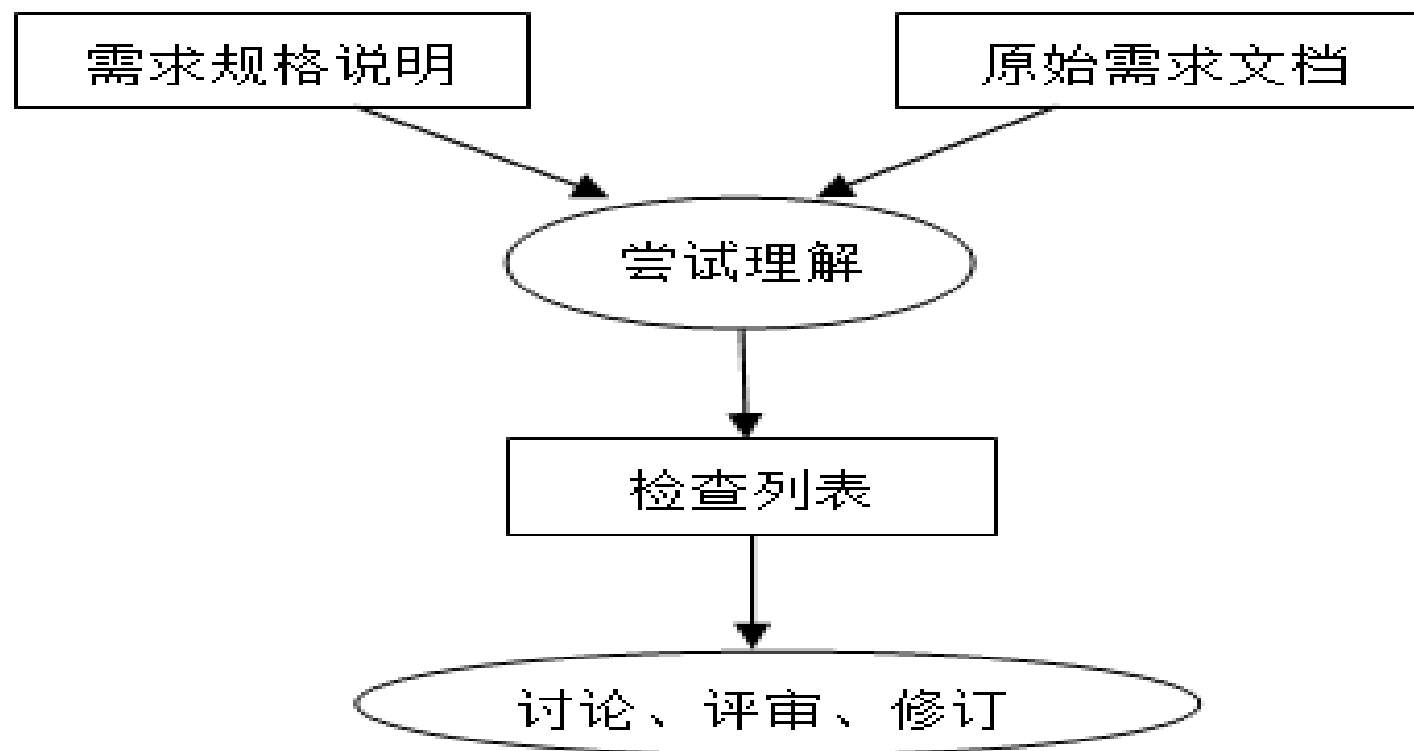
- **正确性**：对照原始需求检查需求规格说明书
- **必要性**：不能回溯到出处的需求项可能是多余的。覆盖的需求应该不多不少，少了则覆盖不充分，多了则可能不必要，检查是否存在多余的需求项是本项检查的重点。
- **优先级**：检查是否恰当地划分并标识需求项。
- **明确性**：不使用含糊的词汇。查找诸如可能、大概、大约等关键字，如果出现这些字，则需要看关键字出现的地方描述的内容是否能进一步地明确。

需求规格说明书检查要点（续）

- **可测性**：每项需求都必须是可验证的，检查是否定义了所有允许的输入，输入格式，如单位、范围、顺序等。是否清晰地定义了某个输入对应的可能输出及输出格式。另外，也包括检查软件系统的约束条件是否完整描述等。
- **完整性**：找到用户的原始需求素材来进行对照检查，包括用户需求文档、用户提供的相关材料、调研记录、与用户的沟通记录等。另外，还包括检查软件使用环境的描述是否清晰等。
- **可修改性**：检查是否具有良好的组织结构，良好的组织结构使需求易于修改。
- **一致性**：主要检查需求是否自相矛盾，包括需求描述的前后不一致和需求规格说明书和原始需求的不一致。

需求规格说明书检查步骤

需求文档的检查步骤如下图所示：



需求规格说明书检查步骤

需求文档的一般检查步骤：

- (1) 获取最新版本的软件需求规格说明书，同时尽量取得用户原始需求文档。
- (2) 阅读和尝试理解需求规格说明书中描述的所有需求项。
- (3) 对照需求规格说明书检查列表进行检查并记录。
- (4) 针对检查结果进行讨论、修订需求规格说明书后回到第一步，直到检查列表中的所有项通过。

需求规格说明书检查步骤

下面是需求规格说明书检查列表的一个例子：



序号	检查项	检查结果	说明
1	是否覆盖了用户提出的所有需求项	是[] 否[] NA[]	
2	用词是否清晰，语义是否存在有歧义的地方。	是[] 否[] NA[]	
3	是否清楚地描述了软件系统需要做什么及不做什么	是[] 否[] NA[]	
4	是否描述了软件使用的目标环境，包括软硬件环境	是[] 否[] NA[]	
5	是否对需求项进行了合理的编号	是[] 否[] NA[]	
6	需求项是否前后一致、彼此不冲突	是[] 否[] NA[]	
7	是否清楚说明了系统的每个输入、输出的格式，以及输入输出之间的对应关系	是[] 否[] NA[]	
8	是否清晰描述了软件系统的性能要求	是[] 否[] NA[]	
9	需求的优先级是否合理分配	是[] 否[] NA[]	
10	是否描述了各种约束条件	是[] 否[] NA[]	



需求规格说明书检查步骤

下面对上表中的每一个检查项做出解释：

- **第一项需要检查需求规格说明书是否满足了用户提出的每一项需求。**因此，需要找到用户的原始需求素材来进行对照检查，包括用户需求文档、用户提供的相关材料、调研记录、与用户的沟通记录等。本项检查的主要是需求的**完整性**。
- **第二项需要检查需求文档的用词用语问题。**因此需要查找诸如也许、可能、大概、大约等关键字，如果出现这些关键字，则需要看关键字出现的地方描述的内容是否能进一步地确定和明确。本项检查的主要是需求的**明确性**。

需求规格说明书检查步骤

- 第三项检查的是需求规格说明书对需求覆盖是否准确。覆盖应该不多不少，少了则是需求覆盖不充分；多了则可能是不必要。检查是否存在多余的需求项是本项检查的重点。本项检查的是需求的**必要性**。
- 第四项检查的是软件使用环境的描述是否清晰。软件使用环境是开发调试和测试的基础，测试人员需要根据这些使用环境进行测试环境的搭建，以便尽量真实地模拟软件将来在用户工作中的使用环境。本项检查的是需求的**完整性**。

需求规格说明书检查步骤

- 第五项检查的是需求的规格说明书中的需求编号是否正确。缺乏需求编号可能导致需求文档的可修改性和可追溯性不强，错误的需求编号则会导致需求维护和管理难度增大。本项检查的主要是需求的**可修改性**。
- 第六项主要检查需求是否是自相矛盾的。包括需求描述的前后不一致和需求规格说明书与用户原始需求的不一致。本项检查的是需求的**一致性**。

需求规格说明书检查步骤

- **第七项主要检查软件系统允许的输入与预期的输出。**是否清晰地定义了所有允许的输入，输入的格式，例如单位、范围、顺序等。是否清晰地定义了某个输入对应的可能输出是什么，以及输出的格式。本项检查的主要是需求的**可测性**。
- **第八项检查的是软件系统的性能需求有没有得到清晰的表述。**实际上不仅仅是性能需求，还可能包括更多的其它需求，例如，如果对产品的安全性有很高的要求了，则需要包括安全性的需求。本项主要检查的是需求的**完整性**。

需求规格说明书检查步骤

- 第九项检查的是需求的关注重点和实现的先后顺序是否清晰地被描述出来。一般软件的关键特性和重要特性应该尽早实现，用户关心的功能应该重点实现，用户迫切想要的功能应该及早提供。本项主要检查的是需求的**优先级**。
- 第十项检查的是对软件系统的约束条件是否完整描述。包括约束条件是否完整、约束条件是否合理，是否与用户的业务场景要求一致等。本项主要检查的是需求的**可测性**。

通过编写测试用例来检查

- 除了针对需求规格说明书进行直接的审查外，还可以通过间接的方式来检验需求的完整性、正确性、可测性等。开发人员通过尝试设计，可以反过来验证需求规则说明书能否很好地指导设计和开发；而测试人员则可以通过设计测试用例来检查需求。
- 测试人员通过想象产品已经制造出来，构建一系列的测试用例，并且问一些“假设”的问题，尝试回答这些测试用例并且与设计人员讨论答案，试图认同答案通常会导致更多的“假设”问题，引发的“假设”问题都必须得到很好的回答，否则可认为需求还不够清晰、完备，或者是可测试性不够强。

通过编写测试用例来检查

- 对于下面一个简单的测试用例，想象软件产品已经存在。

测试用例编号：Input_001

测试优先级：中等

估计执行时间：2 分钟

测试目的：验证业务单据数据的查询正确性。

标题：业务单据查询

步骤：

1. 打开查询界面
2. 输入查询条件
3. 确定并提交查询
4. 查看并验证返回的信息

是否有
疑问？

通过编写测试用例来检查

- 例如，可输入的查询条件包括哪些？提交查询之前是否会验证输入数据的正确性？输入数据的单位、范围有无限制？所有条件都不输入是否意味着查询出所有业务单据？
- 对于这些疑问，如果在需求文档中不能找到令人满意的答案，则可认为需求文档的完整性、正确性、可测试性都存在缺陷。在进一步编写测试用例和测试之前应该解决需求文档的这些问题。

目录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

测试计划

- 测试计划活动一般包括以下8项：

- (1) 定义测试的整体方式和策略。
- (2) 确定测试环境。
- (3) 定义测试级别及它们之间的协作，将测试活动集成到其他项目活动中并进行协调。
- (4) 确定如何评估测试结果。
- (5) 选择监视和控制测试工作的度量，并定义测试出口准则。
- (6) 确定要准备的测试文档，并确定模板。
- (7) 编写测试计划并确定测试的内容、人员、进度和测试范围。
- (8) 估算测试工作量和成本，（再）估计和（再）计划测试任务。

目录

CONTENTS

3.1 测试计划的目标

3.2 制定测试计划

3.3 划分测试用例优先级

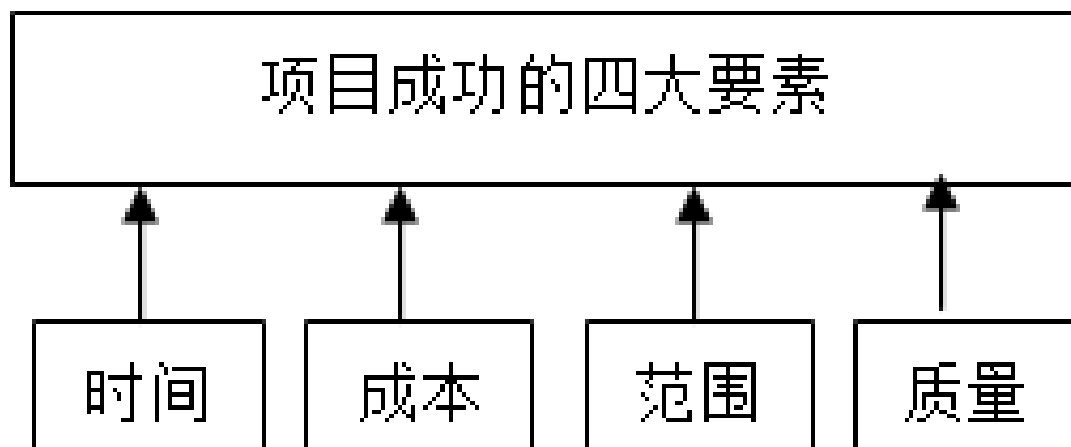
测试计划的目标

● 测试计划所要达到的目标有以下6点:

- (1) 为测试各项活动制订一个现实可行的、综合的计划，内容包括每项测试活动的对象、范围、方法、进度和预期结果。
- (2) 为项目实施建立一个组织模型，并定义测试项目中每个角色的责任和工作内容。
- (3) 开发有效的测试模型，能正确地验证正在开发的软件系统。
- (4) 确定测试所需要的时间和资源，以保证其可获得性和有效性。
- (5) 确立每个测试阶段测试完成及测试成功的标准和要实现的目标。
- (6) 识别出测试活动中的各种风险，并消除可能存在的风险，降低由不可能消除的风险所带来的损失。

测试计划的作用

- 项目的成败由四大要素决定



测试计划的作用

- 项目的4个因素由不同的文档来覆盖
 - 时间：由项目计划覆盖
 - 成本：由合同覆盖
 - 范围：由需求文档覆盖
 - 质量：由QA计划或测试计划覆盖
- 测试计划通常作为关于质量的重要文档呈现给管理层。测试计划具有内部作用和外部作用。

测试计划的作用

- **内部作用**有以下3个：
 - 作为测试计划的结果，让相关人员和开发人员来评审。
 - 存储计划执行的细节，让测试人员来进行同行评审。
 - 存储计划进度表、测试环境等更多的信息。
- **外部作用**是给顾客一个信心，向顾客交代有关于测试的过程、人员的技能、资源、使用的工具 等信息。

测试计划与测试过程的关系

- 软件测试计划是对测试过程的一个整体上的设计。通过收集与项目和产品相关的信息，对测试范围、测试风险进行分析，对测试用例、工作量、资源和时间等进行估算，对测试采用的策略、方法、环境、资源、进度等做出合理的安排。
- 因此，测试计划的要点包括以下内容：
 - 确定测试范围
 - 制定测试策略
 - 确定测试任务
 - 测试资源安排
 - 进度安排
 - 风险及对策

目 录

CONTENTS

3.1 测试计划的目标

3.2 制定测试计划

3.3 划分测试用例优先级

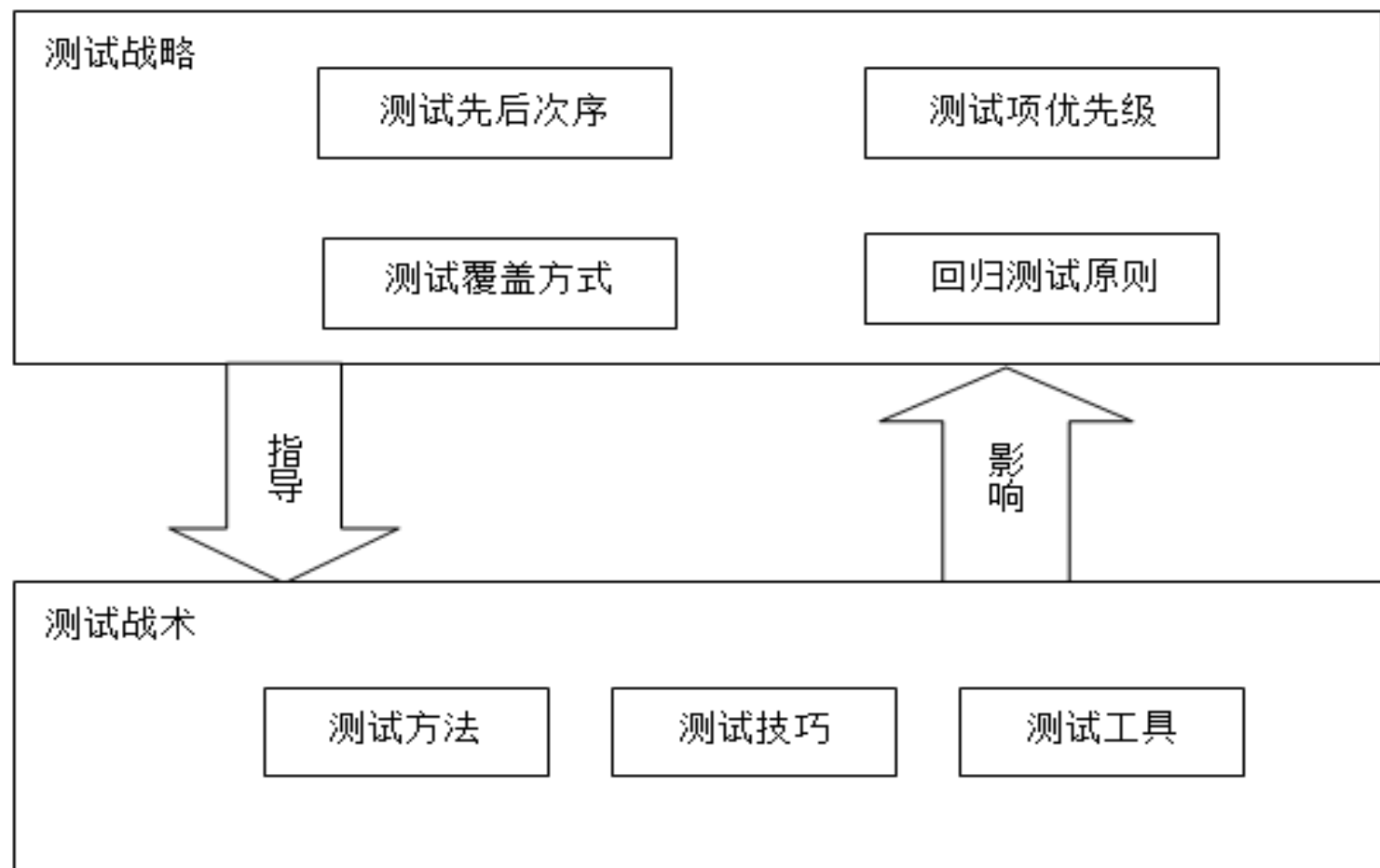
制定测试计划

测试计划的制定过程：

(1) 首先要明确测试的对象，有些对象是不需要测试的，有时候，测试的范围是比较难判断的，例如，对于一些整合型的系统，它把若干个已有的系统整合进来，形成一个新的系统，那么就需要考虑测试的范围是包括所有子系统，还是仅仅测试接口部分，需要结合整合的方式，系统之间通信的方式。

制定测试计划

(2) 制定测试策略，测试的策略包括宏观的测试战略和微观的测试战术。其中，测试战略和测试战术的关系如图所示。



制定测试计划

● 测试战略

- 测试的战略，也就是**测试的先后次序、测试的优先级、测试的覆盖方式、回归测试的原则**等。为了设计出好的测试战略，需要了解软件的结构、功能分布、各模块对用户的重要程度等，从而决定测试的重点、优先次序等。
- 为了达到**有效的覆盖**，需要考虑测试用例的设计方法，尽可能用最少的测试用例发现最多的缺陷，尽可能用精简的测试用例覆盖最广泛的状态空间，还要考虑哪些测试用例使用自动化的方式实现，哪些使用人工方式验证等。
- **回归测试**也需要充分考虑，根据项目的进度安排，版本的迭代频率等，合理安排回归测试的方式，同时也要结合产品的特点，功能模块的重要程度、出错的风险等来制定回归测试的有效策略。

制定测试计划

● 测试战术

- 测试的战术也就是采用的**测试方法、技巧、工具**等。
- 制定测试计划时，需要结合软件采用的技术、架构、协议等，来考虑如何综合各种测试方法和手段，是否需要**进行白盒测试**，采用什么测试工具进行**自动化测试、性能测试**等。

制定测试计划

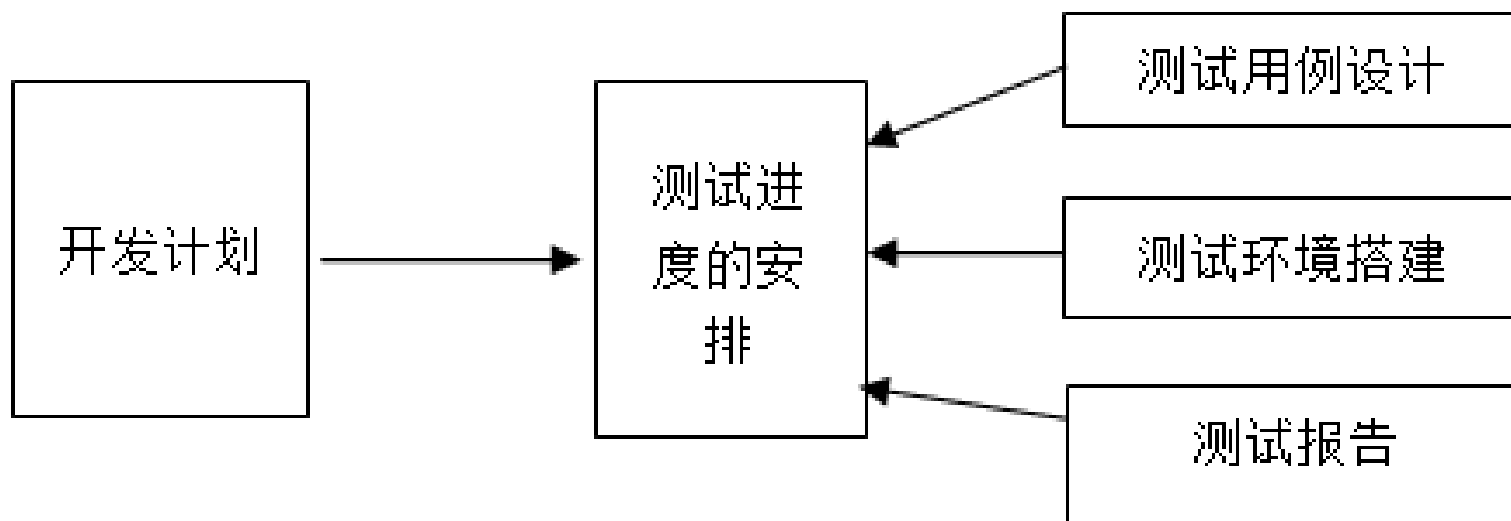
(3) 确定测试任务，根据测试阶段的测试需求，细化测试任务。其中包括划分测试任务的优先级、确定各任务与主要任务之间的关联关系、确定辅助任务清单。

(4) 安排测试资源，通过充分估计测试的难度、测试的时间、工作量等因素，决定测试资源的合理利用。需要根据测试对象的复杂度、质量要求，结合经验数据对测试工作量做出评估，从而确定需要的测试资源。

(5) 安排测试进度，测试的进度安排需要结合项目的开发计划、产品的整体计划进行考虑，还要考虑测试本身的各项活动进行安排。把测试用例的设计、测试环境的搭建、测试报告的编写等活动列入进度安排表。

制定测试计划

- 测试进度的安排**不能完全按照开发计划一一对应**过来，因为有些开发阶段出来的东西是不需要测试的，例如有些模块是基础模块、核心模块，只能进行白盒测试，而这些模块的这种类型测试可能是这个项目的测试活动不需要涉及的，或者是因为测试组没有这样的资源来进行这种类型的测试。



制定测试计划

(6) 估计风险及对策，制定出相应的应对策略。一般可能碰到的风险是**项目计划的变更、测试资源不能及时到位**等方面，制定测试计划时应该根据项目的实际情况进行评估，并制定出合理、有效的应对策略。对于项目计划的变更，可以考虑建立更加通畅的沟通渠道，让测试人员能及时了解到变更的情况，以及变更的影响，从而可以做出相应的改变，例如测试计划的调整。

目 录

CONTENTS

3.1 测试计划的目标

3.2 制定测试计划

3.3 划分测试用例优先级

划分测试用例优先级

● 用于划分测试用例优先级及确定测试用例执行顺序的准则

- (1) 使用频率或失效的概率。系统的某些特定的被经常使用的功能优先级更高。
- (2) 失效的风险。风险是严重性和失效概率的综合结果，高风险失效的用例应该比低风险失效的用例具有更高的优先级。
- (3) 失效的可见性。失效对用户的可见性是划分测试优先级的更进一步准则。
- (4) 需求的优先级。系统提供的不同功能对于客户来说，其重要性也不尽相同。某些功能不能正常工作，客户也许能够接受，但有些功能则不可或缺。

划分测试用例优先级

● 用于划分测试用例优先级及确定测试用例执行顺序的准则

- (5) 除了功能需求，质量特性对于客户也具有不同的重要性。必须测试重要的质量特性是否已经正确实现，并保证用于验证与必要的质量特性是否一致的测试用例具有更高优先级。
- (6) 从系统架构开发人员的角度来确定。失效时会导致严重后果（如系统崩溃）的组件需要加强测试。
- (7) 单独的组件和系统组件的复杂性。复杂的程序组件需要加强测试，因为开发人员可能在其中引入较多的软件缺陷。不过，看起来简单的程序组件也可能会因为开发不够细致而包含很多软件缺陷。因此，对这个领域划分优先级时，应该参考从组织中早期项目得来的经验数据。
- (8) 存在高项目风险的失效应该尽早被发现。这些失效需要做大量的修正工作，否则会独占资源并导致项目明显延迟。
- (9) 项目经理应该为项目定义充分的优先级准则和优先级类别。

目 录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

目 录

CONTENTS

4.1 测试用例的设计原则

4.2 测试用例的设计方法

4.3 测试用例的粒度

4.4 测试用例的评审

测试用例的设计原则

● 设计测试用例时应遵循以下原则

- (1) **正确性**。满足需求规格说明书的要求；覆盖需求规格说明书中的各项功能。
- (2) **全面性**。覆盖所有的需求功能项。
- (3) **整体连贯性**。测试用例执行粒度尽量保持每个用例有一个测试点，不能同时覆盖很多功能点。
- (4) **可维护性**。由于软件开发过程中需求变更等原因的影响，常常需要对测试用例进行修改、增加、删除等。
- (5) **测试结果可判定性和可再现性**。测试结果可判定性是指测试执行结果的正确性是可判定的，每个测试用例都应有相应的期望结果。测试结果可再现性是指对于同样的测试用例，系统的执行结果是相同的。

目 录

CONTENTS

4.1 测试用例的设计原则

4.2 测试用例的设计方法

4.3 测试用例的粒度

4.4 测试用例的评审

测试用例的设计方法

1. 等价类划分法

等价类划分法是一种典型的黑盒测试用例的设计方法。采用等价类划分法时，完全不用考虑程序的内部结构，设计测试用例的唯一依据是软件需求规格说明书。

- **等价类**：输入条件的一个子集合，该集合中的数据对于揭示程序中的错误是等价的。
- **有效等价类**代表对程序有效的输入，而**无效等价类**则是其他任何可能的输入（即不正确的输入值）。
- 有效等价类和无效等价类都是使用等价类划分法设计用例时所必需的，因为被测程序若是正确的，就应该既能接受有效的输入，也能经受住无效输入的考验。

测试用例的设计方法

1. 等价类划分法示例

- 要求：注册用户名要求7~12个字符，可以由字母、数字、下画线构成；下画线不能作为开头。该示例的等价类划分如下表所示。

输入条件	有效等价类	无效等价类
用户名	7~12个字符 (1)	少于7个字符 (2) 多于12个字符 (3) 空 (4)
用户名	由字母、数字、下画线构成 (5)	含有除字母、数字、下画线以外的特殊字符 (6) 非打印字符 (7) 中文字符 (8)
	以字母、数字开头 (9)	以下画线开头 (10)

测试用例的设计方法

2. 边界值分析法

边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。

3. 基本路径分析法

基本路径分析法是在程序控制流图的基础上，通过分析控制构造的圈复杂性，导出基本可执行路径集合，从而设计测试用例的方法。利用此方法设计出的测试用例要保证在测试程序的每个可执行语句中都至少执行一次。

4. 因果图法

因果图法是一种利用图解法分析输入的各种组合情况，从而设计测试用例的方法。它适合于检查程序输入条件的各种组合情况。

目 录

CONTENTS

4.1 测试用例的设计原则

4.2 测试用例的设计方法

4.3 测试用例的粒度

4.4 测试用例的评审

测试用例的粒度

- 测试用例可以写得很简单，也可以写得很复杂。
- **最简单的测试用例**是测试的纲要，仅仅指出要测试的内容，如探索性测试中的测试设计，仅会指出需要测试产品的哪些要素、需要达到的质量目标、需要使用的测试方法等。
- **最复杂的测试用例**会指定输入的每项数据，期待的结果及检验的方法，具体到界面元素的操作步骤，指定测试的方法和工具等。

测试用例的粒度

- **测试用例写得过于复杂或详细**，会带来两个问题：一个是效率问题，另一个是维护成本问题。另外，测试用例设计得过于详细，留给测试执行人员的思考空间就比较少，容易限制测试人员的思维。
- **测试用例写得过于简单**，则可能失去了测试用例的意义。过于简单的测试用例设计其实并没有进行“设计”，只是把需要测试的功能模块记录下来而已，它的作用仅仅是在测试过程中作为一个简单的测试计划，提醒测试人员测试的主要功能包括哪些而已。
- **测试用例的设计的本质**应该是在设计的过程中理解需求，检验需求，并把对软件系统的测试方法的思路记录下来，以便指导将来的测试。

测试用例的粒度

● 测试用例的设计粒度需要考虑以下4个方面的因素

- (1) **复用率**：如果产品在不断地更新版本，复用率很高，测试用例的粒度需要更加细化；相反，如果产品的使用频率不高，那么测试用例就不需要设计得很复杂。
- (2) **项目进展**：需要根据项目的进展而定，如果当前项目的时间充足，可以将测试用例设计得详细一些，但是如果项目距离截止时间较近，没有多余的时间，则需要将测试用例设计得简单一些。
- (3) **使用对象**：如果所设计的测试用例在测试过程中是提供给多个测试人员使用，需要将测试用例设计得详细一些。
- (4) **测试的种类**：如果采用验收测试，那么相应的测试用例的粒度就比较大。如果是系统测试，那么测试用例的颗粒度就相对较小。

目 录

CONTENTS

4.1 测试用例的设计原则

4.2 测试用例的设计方法

4.3 测试用例的粒度

4.4 测试用例的评审

测试用例的评审

如何提高测试用例的设计质量？

- 测试用例的检查可以有多种方式，但是最敏捷的应当属临时的同行评审。同行评审尤其是临时的同行评审，应该演变成类似结对编程一样的方式。从而体现敏捷的“个体和交互比过程和工具更有价值”这一原则，要强调测试用例设计者之间的思想碰撞，通过讨论、协作来完成测试用例的设计。
- 原因很简单，测试用例的目的是尽可能全面地覆盖需求，而测试人员总会存在某方面的思维缺陷，一个人的思维总是存在局限性。因此需要一起设计测试用例。

测试用例的评审

- 除了同行评审，还应该尽量引入用户参与到测试用例的设计中来，
让用户参与评审，从而体现敏捷的“顾客的协作比合同谈判更有价值”这一原则。这里顾客的含义比较广泛，关键在于如何定义测试：
 - 如果测试是对产品的批判，则顾客应该指最终用户或顾客代表；
 - 如果测试是被定义为对开发提供帮助和支持，那么顾客显然就是程序员了。

测试用例的评审

● 下面是测试用例评审的一些检查项

- (1) 测试用例是否是按照公司定义的模板进行编写的。
- (2) 测试用例本身的描述是否清晰，是否存在二义性。
- (3) 测试用例内容是否正确，是否与需求目标相一致。
- (4) 测试用例的期望结果是否确定、唯一。
- (5) 测试用例的操作步骤与描述是否相一致。
- (6) 测试用例是否覆盖了所有的需求。
- (7) 测试用例的设计是否存在冗余性。
- (8) 测试用例是否具有可执行性。
- (9) 是否从用户层面来设计用户使用场景和业务流程的测试用例。

目 录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

目录

CONTENTS

5.1

测试用例的合理选择与
测试分工

5.2

测试环境的搭建

5.3

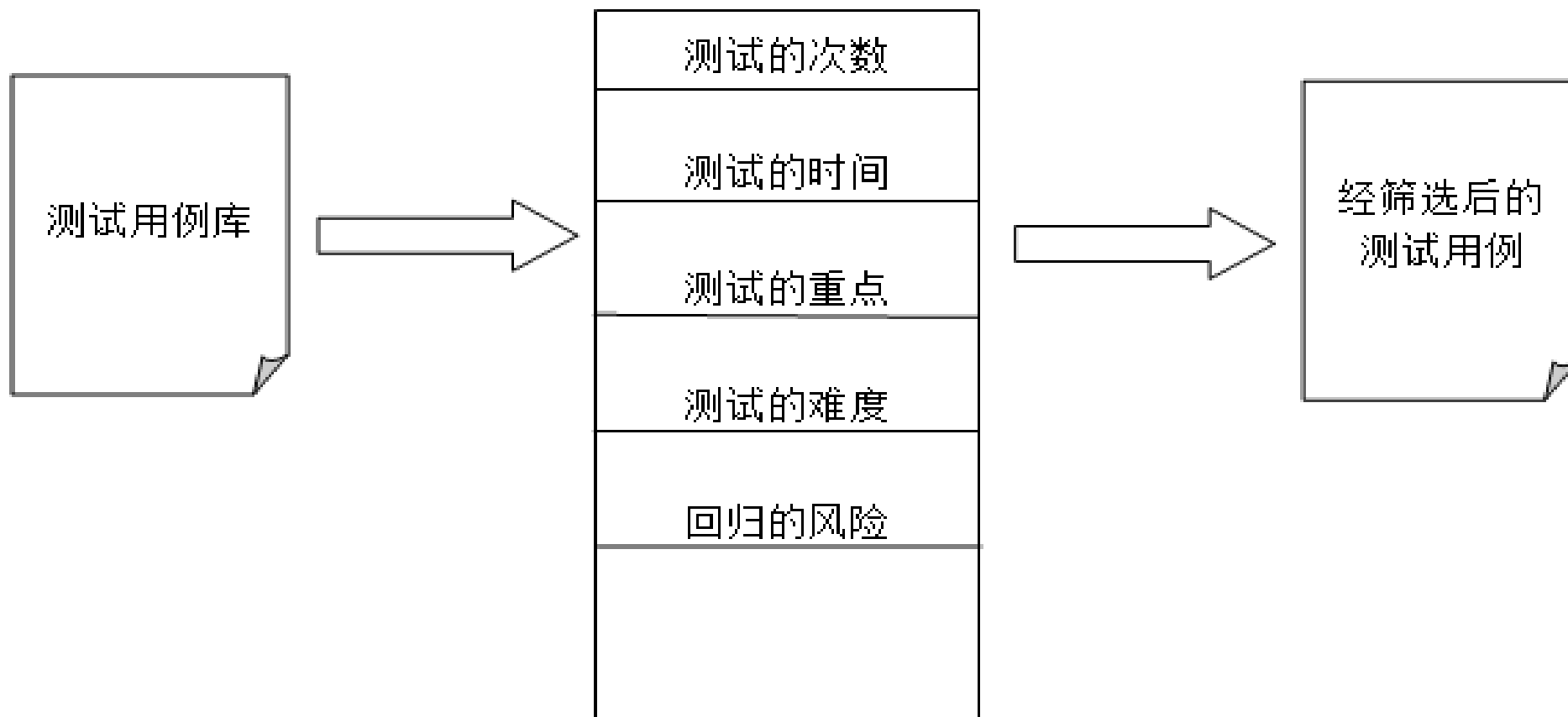
BVT测试与冒烟测试

5.4

每日构建

测试用例的选择

- 测试用例的选择是一个战术问题，是一个考验测试人员智慧的过程，如图所示：



测试用例的选择

- 测试用例的选择需要考虑本次测试的上下文，是第一次测试，还是回归测试？测试持续的时间有多长？自动化脚本的准备情况怎么样了？如果等到最后才做性能测试，是否会加大修改的难度？
- 对于第一次执行的测试，一个基本的测试用例选择策略是：先执行基本的测试用例，再执行复杂的测试用例；先执行优先级高的测试用例，再执行优先级低的测试用例。
- 对于回归测试的测试用例选择则复杂一点，因为大部分测试人员不想花费太多的时间和精力在一些已经执行过的测试用例上，但是又害怕程序员修改的地方会引发已经稳定的模块问题。

测试用例的选择

● 下面是一些测试用例的选择策略

- (1) 首先测试产品的核心功能，再测试其他功能
- (2) 若产品具有支付交易功能，则需要首先测试此功能，再测试其他功能，因为资金的问题永远是最重大的问题。
- (3) 首先测试常用功能，再测试其他功能。
- (4) 首先测试需求中被特别说明的地方，再测试没有特别说明的地方
- (5) 首先测试有变更的地方，后测试没有变更的地方

测试人员分工

(1) 按照测试内容分工

- 一个项目的测试包括文档测试、易用性测试、逻辑功能测试、界面测试、配置和兼容等多个方面,管理者可以根据人员的特点为每个人员分配不同的测试内容。这样分配的优点在于分工较为明确,测试人员对于测试内容的重点具有清楚的认识。

(2) 按照测试流程分工

- 项目的测试流程一般包括测试需求的检查、测试用例的设计、测试执行、输出测试报告等工作,管理者可以根据测试流程中的各个阶段来进行人员的分工。这样分配具有流程清晰的特点。

(3) 按照功能模块分工

- 一般规模较大的软件项目所具有的功能模块较多,针对已划分好的功能模块给予相应的测试,不同的测试人员负责不同模块的测试工作。这样分配具有人员利用率高且容易发现深层错误的特点。

(4) 按照测试类型分工

- 软件测试根据软件开发的阶段可以划分为单元测试、集成测试、系统测试、回归测试等测试类型,管理者可以根据这些类型为测试人员分配测试工作。这样分配具有对测试人员的专业性要求高的特点。

测试人员分工

- 测试的分工能**避免测试人员的思维局限性**，即使是同样一个用例，由不同的人来执行，可能会发现不同的问题。
- 如果测试组由新人和有经验的测试工程师搭配，则**分工可以帮助新人更快地成长**。
- 除了测试组本身的测试资源的利用之外，还要**合理寻找和请求其它的测试资源**。
- 如果公司有多个项目，则可能出现不同的项目进度不一样、所处的阶段不一样，因此也可以**请其他比较空闲的项目组的测试人员协助测试**。

目 录

CONTENTS

5.1

测试用例的合理选择与
测试分工

5.2

测试环境的搭建

5.3

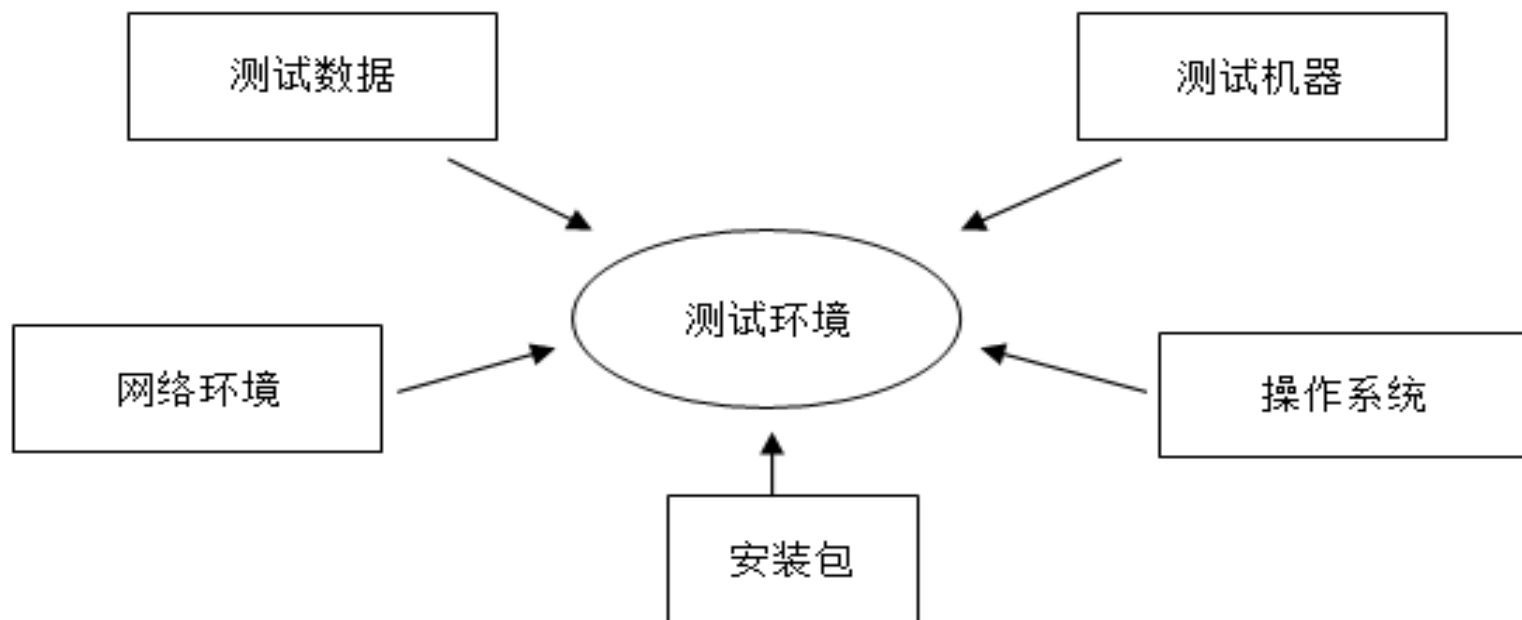
BVT测试与冒烟测试

5.4

每日构建

测试环境的搭建

- 测试环境的搭建在某些项目的测试过程中是一个**非常重要**的工作，同时也可能是一项**很耗时**的工作。有些软件的测试环境要求比较复杂，需要在测试执行之前做好充分的准备。一般测试环境的搭建包括下图所示的内容：



测试环境的搭建

- 有些测试**需要使用大批量的数据**，例如容量测试、压力测试等。
根据产品的具体测试要求，可能需要在数据库表插入大量的数据，准备大量的文件、生成大量的Socket包等。
- 有些测试**需要使用专门的外部硬件设备**，例如打印机、条码识别器、读卡机、指纹仪等。如果是手机的应用测试，则可能要把所有支持的型号的手机都要准备好。这些设备有些可以使用模拟器来模拟，有些则不能。

测试环境的搭建

- 有些产品**需要支持多种操作系统**，那么在做兼容性测试之前就需要准备好包含各种操作系统的计算机，或者考虑使用虚拟操作系统工具来安装多个操作系统，例如VM Ware、Virtual PC等。
- 有些测试**需要部署到多台机器**，并且**需要设置各种参数**，那么就需要在测试之前准备好各种安装包。
- 有些测试**需要用到网络**，设置需要考虑网络的路由设置、拓扑结构等，那么在测试之前就要准备好这样的网路设备和网络环境配置。

目 录

CONTENTS

5.1

测试用例的合理选择与
测试分工

5.2

测试环境的搭建

5.3

BVT测试与冒烟测试

5.4

每日构建

BVT测试

- 构建验证测试（Build Verification Test, BVT），也叫编译检查测试，主要检查源代码是否能正确编译成一个新的、完整可用的版本。BVT是在所有开发工程师都已经检查完自己的代码，项目组编译生成当天的版本之后进行。如果BVT测试不通过，则测试人员不能拿到新的版本进行测试。
- BVT的优点是时间短，验证了软件的基本功能。
- BVT的缺点是测试的覆盖率很低。因为运行时间短，不可能把所有情况都测试到。

冒烟测试

- 冒烟测试，是对软件基本的功能进行测试，测试的对象是每个新编译的需要正式测试的软件版本。
- 目的是确认软件基本的功能是否正常，保证软件系统能够正常运行，并且可以进行后续的正式测试工作。如果最基本的测试都有问题，就需要向开发人员进行反馈，所以正式交付测试的版本首先必须通过冒烟测试的考验。
- 冒烟测试，只是一个测试活动，并不是一个测试阶段。也就是说，冒烟测试贯穿于测试的任何一个阶段，单元测试里会有冒烟测试、集成测试里会有冒烟测试、系统测试里也会有冒烟测试。

BVT测试与冒烟测试的区别

- BVT测试与冒烟测试的**不同点**如下：
 - BVT只在构建完成时进行；冒烟测试是各个阶段都会进行。
 - 尽管 BVT 可以加入自动测试脚本并执行少量固定的自动化测试，但冒烟测试与 build 的验证无关。
 - BVT的结果直接决定新构建的build是否交付后续测试；冒烟测试不影响其他日常测试工作。
- BVT测试与冒烟测试的目的是检查程序是否完整，是否实现了最基本的可测试性要求。能有效减少测试人员的不必要的工作量。

BVT测试与冒烟测试是所有正式测试执行之前的第一个步骤。

目 录

CONTENTS

5.1

测试用例的合理选择与
测试分工

5.2

测试环境的搭建

5.3

BVT测试与冒烟测试

5.4

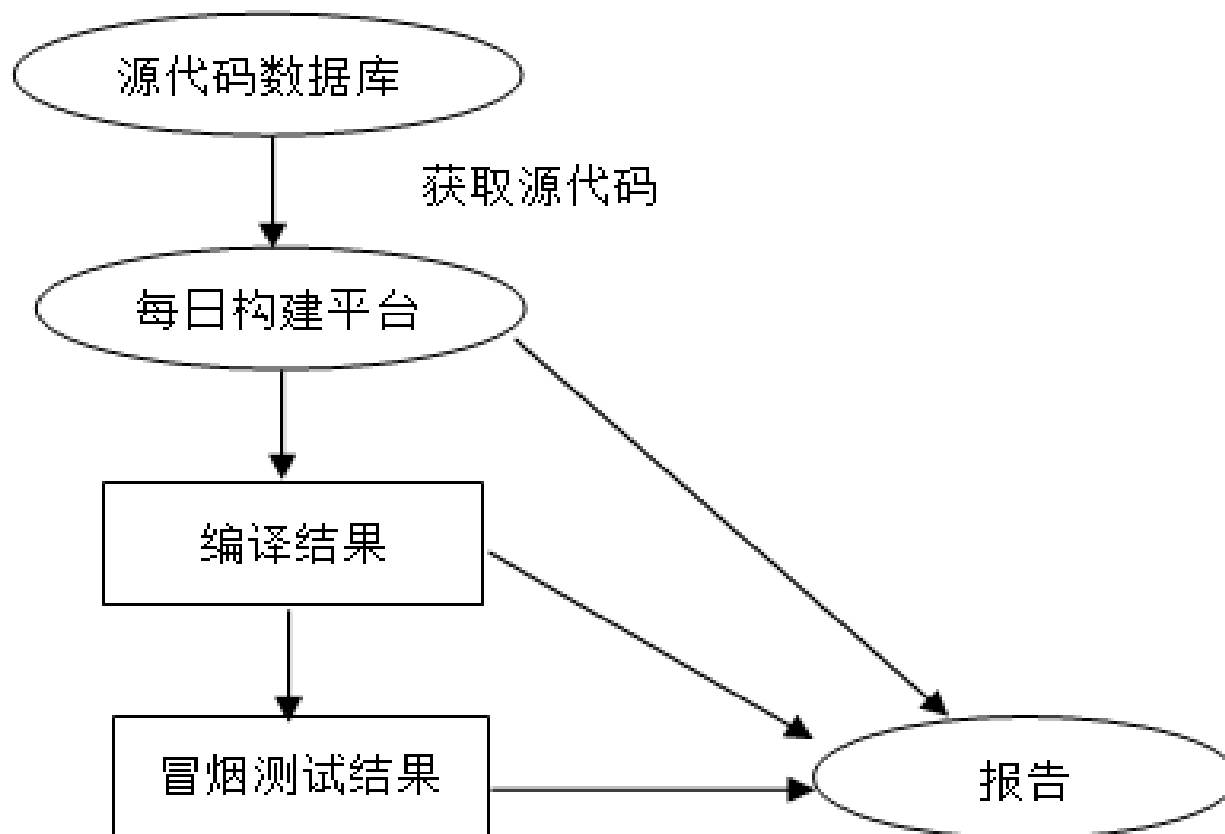
每日构建

每日构建

- 程序模块的集成问题是一个导致开发进度受阻的常见原因。
- 缺陷也往往在集成阶段才集中出现，尤其是那些接口设计不够好的软件。
- 每日构建就是每天定时把所有文件编译、连接、组成一个可执行的程序的过程。
- 通常把每日构建放在晚上，利用空余时间自动进行，因此也叫每晚自动构建。

每日构建的基本流程

- 每日构建也可称为**持续集成（Continuous Integration）**。它强调完全自动化的、可重复的创建过程，其中包括每天运行多次的自动化测试。



每日构建的优点

- 进度可见并可以控制到1-2天的细粒度，很容易**看到**进度的偏差。
- 可以尽早发现软件缺陷并分析解决，从而**提高**软件质量。
- 由于将大集成分解到每日构建中的小集成，**消除**了传统产品集成或集成测试时出现严重问题的可能性。
- 注重每次工作的正确性，**减少**了可能出现的错误。

目 录

CONTENTS

01

软件测试的各个阶段

02

测试需求

03

测试计划

04

测试设计及测试用例

05

测试的执行

06

软件缺陷分析

目 录

CONTENTS

6.1 软件缺陷分析的作用

6.2 软件缺陷的分类

6.3 软件缺陷分析方法

6.4 软件缺陷分析的流程

6.5 软件缺陷报告

6.1 软件缺陷分析的作用

- 软件缺陷所具有的含义不仅仅局限于程序中存在的软件缺陷。
- 软件测试的任务就是发现软件系统的缺陷，保证软件的优良品质。
- 软件缺陷分析对于改进软件开发、提高软件质量有着十分重要的作用。

6.2 软件缺陷的分类

(1) 按严重程度划分

按照严重程度由高到低的顺序可以将软件缺陷分为5个等级：**系统崩溃、严重、一般、次要、建议**。在具体的项目中，需要根据实际情况来划分等级，不一定是5个等级。如果软件缺陷数比较少，就可以划分为3个等级：**严重、一般、次要**。一般的软件缺陷管理工具会自动给出一个默认的软件缺陷严重程度划分。

(2) 按优先级划分

按照缺陷修复的优先级由高到低可以划分为3个等级：**高（high）、中（middle）、低（low）**。其中，高优先级的软件缺陷是应该立即修复的软件缺陷；中优先级的软件缺陷是应该在产品发布之前修复的软件缺陷；低优先级的软件缺陷是指如果时间允许，应该修复的软件缺陷或是可以暂时存在的软件缺陷。

6.2 软件缺陷的分类

(3) 按测试种类划分

按测试种类可以将软件缺陷分为逻辑功能类、性能类、界面类、易用性类、兼容性类。这种划分方式可以了解不同测试方法所能发现的软件缺陷的比例，以便在测试的时候有所侧重。

(4) 按功能模块划分

一般的软件产品都是分为若干个功能模块的，根据二八定理可知，通常80%的软件缺陷集中在20%的模块中，测试的过程可以统计软件缺陷主要集中在哪些模块，以便投入重点精力去测试。

6.3 软件缺陷分析方法

(1) 正交缺陷分类法 (Orthogonal Defect Classification, ODC)

该方法是将一个软件缺陷在生命周期的各环节的属性组织起来，从单维度、多维度对软件缺陷进行分析，从不同角度得到各类软件缺陷的缺陷密度和缺陷比率，从而积累得到各类软件缺陷的基线值，用于评估测试活动。

(2) 冈珀茨 (Gompertz) 法

该方法是在利用已有数据的基础上，对软件测试过程进行定量分析和预测，对软件产品质量进行定量评估，对是否结束软件测试任务给出判断依据。

6.3 软件缺陷分析方法

(3) DRE/DRM分析方法

DRE (Defect Removal Efficiency) 分析方法是通过已有项目历史数据，得到软件生命周期各阶段缺陷注入和排除的模型，用于设定各阶段质量目标，评估测试活动。

DRE主要针对历史数据，矩阵的每一列代表软件缺陷在何时（什么阶段）引入（产生），每一行代表发现软件缺陷时开展的工作。矩阵中的数值代表已经发现的软件缺陷数量。

6.3 软件缺陷分析方法

(3) DRE/DRM分析方法

下表所示的矩阵的目标是要**分别计算出各个阶段的软件缺陷移除率**为后面所用。软件缺陷移除率的定义为当前阶段工作实际发现的软件缺陷数量占当前阶段应该发现的软件缺陷数量的比值。

发现阶段 \ 注入阶段	需求	设计	编码	发现总计
需求阶段	8	—	—	8
设计阶段	26	62	—	88
编码阶段	4	11	12	27
功能测试阶段	4	3	112	119
系统测试阶段	0	0	28	28
注入总计	42	76	152	270

6.4 软件缺陷分析的流程

(1) 确定分析指标

①反映产品质量的指标:

软件缺陷密度 = 软件缺陷数量 / 软件规模

潜在软件缺陷数 = (100% - 发布前软件缺陷去除率) × 软件缺陷密度

②反映产品可靠性的指标:

平均失效时间 = 软件持续运行时间 / 软件缺陷数量

③反映软件缺陷发现及修复效率的指标:

软件缺陷检出率 = 某阶段当时发现的软件缺陷 / 属该阶段的全部软件缺陷 × 100%

发布前软件缺陷去除率 = 发布前发现的软件缺陷 / (发布前发现的软件缺陷 +
软件运行的前3个月发现的软件缺陷) × 100%

软件缺陷修正率 = 修复过程中未引发其他问题的软件缺陷数 / 被修复软件缺陷的总数 × 100%

6.4 软件缺陷分析的流程

(1) 确定分析指标

④反映软件缺陷修复成本的指标:

$$\text{平均修复时间} = \frac{\sum \text{软件缺陷修复时间}}{\text{软件缺陷数量}}$$

$$\text{平均修复成本} = \text{开发人员的平均人力成本} \times \text{平均修复时间}$$

$$\text{相对返工成本} = \text{返工的工作量} / \text{项目总工作量} \times 100\%$$

6.4 软件缺陷分析的流程

(2) 实施软件缺陷分析过程

在确定好相应的指标之后，可以借助相应的方法和工具实施软件缺陷分析的过程，产生分析的结果。

(3) 汇总统计

- 软件缺陷性质统计：变更性质属性一般分为软件缺陷变更和需求变更两种。
- 软件缺陷状态分布：变更状态属性分类很多，但在软件缺陷分析中可以不用分得很细，可以分为关闭、挂起和处理中3种类型。该分析主要反映软件缺陷修改完成情况。
- 软件缺陷按产品分类统计：该分析能显示各软件子系统的软件缺陷分布情况。

6.4 软件缺陷分析的流程

(3) 汇总统计

- **软件缺陷按原因分类统计：**按变更的根本原因属性进行分类统计软件缺陷，统计不包括需求变更。该分析能揭示软件缺陷原因的分布。
- **软件缺陷测试情况统计：**统计仅涉及变更的根本原因是系统设计、程序编码、维护和外部问题等软件缺陷变更。该分析能暴露软件测试本身存在的问题。
- **软件缺陷来源统计：**该分析主要反映用户或软件代理的地区分布，发现一些客户分布规律。

6.5 软件缺陷报告

一般情况下，软件缺陷报告包含下列内容：

- **软件缺陷报告编号**：为了便于对软件缺陷进行管理，每个软件缺陷都必须被赋予唯一的编号，编号规则可根据需要和管理要求制订。
- **标题**：软件缺陷报告的标题可以用简明的方式传达软件缺陷的基本信息。软件缺陷报告的标题应该简短并尽量做到唯一，以便在观察缺陷列表时，可以很容易注意到。
- **报告人**：软件缺陷报告的原始作者，有时也可以包括软件缺陷报告的修订者。当负责修复该软件缺陷的开发人员对报告有任何异议或疑义时，可以与报告人进行联系。
- **报告日期**：软件缺陷报告首次报告的日期。让开发人员知道创建软件缺陷报告的日期是很重要的，因为有可能这个软件缺陷在前面版本曾经修改过。

6.5 软件缺陷报告

一般情况下，软件缺陷报告包含下列内容：

- **程序（或组件）的名称**：程序（或组件）的名称可用来分辨被测对象。
- **版本号**：测试可能跨越多个软件版本，提供版本信息可以方便地进行软件缺陷管理。
- **配置**：发现软件缺陷时的软件和硬件配置。如操作系统的类型、是否有浏览器载入、处理器的类型和速度、内存大小、可用内存、正在运行的其他程序等。
- **软件缺陷的类型**：软件缺陷的类型包括代码错误、设计问题、文档不匹配等。
- **严重性**：描述所报告的软件缺陷的严重性。

6.5 软件缺陷报告

一般情况下，软件缺陷报告包含下列内容：

- **优先级**：由开发人员或者管理人员确定软件缺陷的优先级，根据修复这个软件缺陷的重要性而定。
- **关键词**：关键词可在任何时候添加，以便分类查找软件缺陷报告。
- **软件缺陷描述**：对发现的问题进行详细说明，尽管描述要深入，但是简明仍是最重要的。软件缺陷描述的主要目的是说服开发人员决定去修复这个软件缺陷。
- **重现步骤**：重现步骤必须是有限的，并且描述的信息足够使读者知道正确的执行就可以重现这个软件缺陷。
- **结果对比**：在执行了重现软件缺陷步骤后，期望发生什么，实际上又发生了什么。

小结

这个知识点主要学习内容：

- 软件测试是软件开发中的最后一个阶段。
- 软件测试是使用人工或者自动手段来运行或测试某个系统的过程，通过测试发现软件开发设计的过程中存在的问题，其目的在于检验它是否满足规定的需求或弄清预期结果与实际结果之间的差别。
- 软件测试的过程主要描述了软件测试需要做的工作，随着软件测试技术的进步，测试过程也会得到进一步改进。

习题

1. 软件测试一般分为哪几个阶段？
2. 需求规格说明书的检查要点有哪些？
3. 软件质量差的两大原因？
4. 说明测试执行过程中所要做的主要工作。